Skill

Topic 1 : To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
vennela_07@VennelaPC:~$ mkdir OSSP
cd OSSP
vennela_07@VennelaPC:~/OSSP$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/vennela_07/OSSP/.git/
vennela_07@VennelaPC:~/OSSP$ mkdir src include obj bin docs tests scripts assets
vennela_07@VennelaPC:~/OSSP$ touch .gitignore README.md LICENSE Makefile
vennela_07@VennelaPC:~/OSSP$ touch src/{main.c,shell.c,parser.c,executor.c,builtin.c}
vennela_07@VennelaPC:~/OSSP$ touch include/{shell.h,parser.h,executor.h,builtin.h}
vennela_07@VennelaPC:~/OSSP$ touch bin/my_shell
vennela_07@VennelaPC:~/OSSP$ touch docs/{design.md,report.pdf}
vennela_07@VennelaPC:~/OSSP$ touch tests/{test_parser.c,test_executor.c}
vennela_07@VennelaPC:~/OSSP$ touch scripts/run.sh
chmod +x scripts/run.sh
vennela_07@VennelaPC:~/OSSP$ touch assets/architecture.png
vennela_07@VennelaPC:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 19 files
```

Topic 2 : To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

```
vennela_07@VennelaPC:~/OSSP$ cd ~/OSSP
mkdir -p process
cd process
nano fork_demo.c
mkdir: cannot create directory ‘process’: File exists
-bash: cd: process: Not a directory
```

vennela_07@VennelaPC: ~/OS

GNU nano 7.2 fork_demo.c *

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

```
vennela_07@VennelaPC:~/OSSP$ cd ~/OSSP
mkdir -p process
cd process
nano fork_demo.c
mkdir: cannot create directory ‘process’: File exists
-bash: cd: process: Not a directory
vennela_07@VennelaPC:~/OSSP$ gcc fork_demo.c -o fork_demo
vennela_07@VennelaPC:~/OSSP$ ./fork_demo
Parent Process
Parent PID = 601
Child PID = 602
Child Process
Child PID = 602
Parent PID = 285
```

Topic 3 : To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop.

```
vennela_07@VennelaPC:~/week3$ gcc task2.c -o task2
vennela_07@VennelaPC:~/week3$ ./task2
File copied successfully.
vennela_07@VennelaPC:~/week3$ cat File2.txt
This is File1.
This content will be copied to File2.
vennela_07@VennelaPC:~/week3$ ./task2
File copied successfully.
vennela_07@VennelaPC:~/week3$ cat File2.txt
This is File1.
This content will be copied to File2.
vennela_07@VennelaPC:~/week3$ nano task3.c
vennela_07@VennelaPC:~/week3$ gcc task3.c -o task3
vennela_07@VennelaPC:~/week3$ ./task3
Enter command: hello
You entered: hello
```

vennela_07@VennelaPC: ~/we

GNU nano 7.2 task1.c

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while(1)
    {
        printf("myshell$ ");

        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if(strcmp(command, "exit") == 0)
        {
            break;
        }

        printf("You entered: %s\n", command);
    }

    printf("Shell exited.\n");

    return 0;
}
```

Topic 4 : To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
vennela_07@VennelaPC:~/keyboard_project$ mkdir keyboard_project
vennela_07@VennelaPC:~$ cd keyboard_project
vennela_07@VennelaPC:~/keyboard_project$ nano keyboard.c
vennela_07@VennelaPC:~/keyboard_project$ gcc keyboard.c -o keybo
ard
vennela_07@VennelaPC:~/keyboard_project$ ./keyboard
Enter a command (type EXIT to quit):
hello
Command entered: hello
open file
Command entered: open file
exit
Command entered: exit
```

```
 GNU nano 7.2                    keyboard.c *
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[100];
    int ch, i = 0;

    printf("Enter a command (type EXIT to quit):\n");

    while (1) {
        ch = getchar();

        if (ch == '\n') {
            buffer[i] = '\0';

            if (strcmp(buffer, "EXIT") == 0) {
                printf("Exiting...\n");
                break;
            }

            printf("Command entered: %s\n", buffer);
            i = 0;
        }
        else if (ch == 127 || ch == 8) {
            if (i > 0) {
                i--;
            }
        }
        else if (i < 99 && ch >= 32 && ch <= 126) {
            buffer[i++] = ch;
        }
    }

    return 0;
}
```